Internal Neighbor-System Specification

Shared Cutoff, Minimum-Image, and CSR Neighbor Primitives for mdstats

mdstats

2026-07-22

1. Purpose and status

This document specifies the shared neighbor-search layer used by structural analysis modules in `mdstats`.

Public cutoff value objects live in

`mdstats/analysis/cutoffs.py`

and the private numerical kernel lives in

`mdstats/analysis/_neighbors.py`

The low-level facade remains private implementation infrastructure. Stage S0 hardened the blocked dense implementation as the authoritative reference backend. Stage S1 added an exact single-frame triclinic cell-list backend in `mdstats/analysis/_cell_list.py`. Stage S2 added request-keyed fixed-cell candidate reuse, stage S3 added explicit deformation-aware reuse, and stage S4 added the public `NeighborSearchOptions` policy layer in `mdstats/analysis/neighbor_search.py`. RDF, coordination, bond-angle, and distance-based connectivity now share this exact execution policy.

The module centralizes geometry and counting rules needed by:

- pair radial-distribution functions;
- integer coordination distributions;
- bond-angle distributions;
- future radial-angular, local-order, ring, and environment descriptors.

Observable-specific normalization remains outside the neighbor layer. The neighbor system answers only:

> Which selected atoms are within a fixed pair cutoff, and what are their minimum-image displacement vectors and distances?

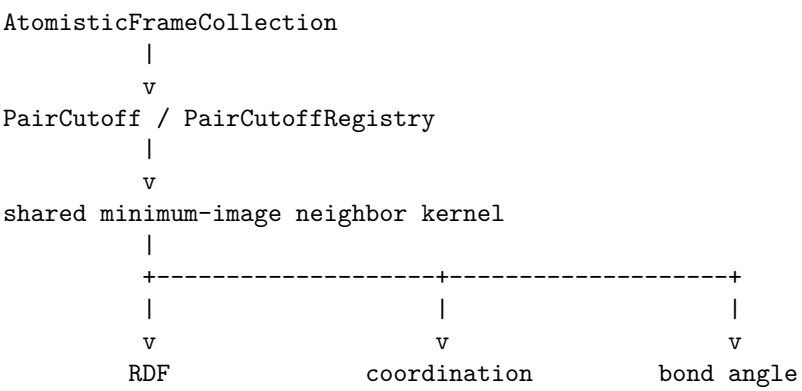
2. Design motives

RDF, coordination, and angle calculations must use identical definitions of:

- atom selection;
- frame selection;
- periodic minimum-image geometry;
- triclinic-cell handling;
- cutoff validation;
- self-pair exclusion;
- cutoff inequality.

Duplicating these rules would allow the same pair to be counted as coordinated in one module and omitted in another. A narrow shared layer prevents this class of inconsistency while leaving each observable free to apply its own pair-counting and normalization.

The architecture is



3. Mathematical conventions

3.1 Cell convention

For frame t, the three lattice vectors are rows of

$$H_t = \begin{pmatrix} \mathbf{a}_t^{\mathsf{T}} \\ \mathbf{b}_t^{\mathsf{T}} \\ \mathbf{c}_t^{\mathsf{T}} \end{pmatrix}.$$

Wrapped Cartesian positions are reconstructed from wrapped fractional coordinates by

$$\mathbf{r}_{i,t} = \mathbf{s}_{i,t} H_t.$$

The neighbor layer operates frame by frame. It does not assume temporal continuity and is valid for trajectories, ensembles, and single-frame collections.

3.2 Minimum-image displacement

For center atom i and candidate neighbor j, define the Cartesian minimum-image vector

$$\mathbf{d}_{ij} = \mathrm{MIC}\left(\mathbf{r}_j - \mathbf{r}_i; H_t, \mathbf{p}\right),$$

where $\mathbf{p}$ is the three-component periodic-boundary mask.

The distance is

$$r_{ij} = \|\mathbf{d}_{ij}\|.$$

The implementation must support orthogonal and triclinic cells. The same minimum-image convention must be used by every structural analysis module.

3.3 Exact unique minimum-image radius

The first-image neighbor contract is valid while the cutoff is no larger than one half of the shortest nonzero translation in the periodic lattice. Let

$$\Lambda_{\mathrm{PBC}}(H) = \left\{ \mathbf{n}H : \mathbf{n} \in \mathbb{Z}^3,\ n_\alpha = 0 \text{ on nonperiodic axes} \right\}.$$

For one frame, mdstats defines

$$r_{\mathrm{safe}}(H) = \frac{1}{2} \min_{\ell \in \Lambda_{\mathrm{PBC}}(H) \setminus \{0\}} \|\ell\|.$$

For several selected frames, the global admissible radius is

$$r_{\mathrm{safe,global}} = \min_t r_{\mathrm{safe}}(H_t).$$

This is the exact inradius of the periodic Wigner–Seitz cell and is the necessary bound for guaranteeing a unique image whenever the retained distance is strictly below the cutoff. It is not generally equal to one half of the shortest cell-vector length, and it is not the historical perpendicular face-height bound.

mdstats obtains the shortest translation from ASE's low-dimensional Minkowski reduction. ASE documents that the reduced basis has the shortest possible vector lengths ordered by norm; its implementation follows Nguyen and Stehle [2]. mdstats validates the returned integer unimodular transform, preservation of the periodic sublattice, transform consistency, and the Minkowski-reduced certificate before accepting the result.

For a fully nonperiodic collection, `compute_safe_cutoff()` returns infinity. Perpendicular cell heights remain useful for cell-list bin sizing and metric stencils, but they are no longer used as the scientific minimum-image cutoff.

For the approximately 60° rhombohedral LTA primitive cell with $\|\mathbf{a}_i\| \simeq 17.36$ A, the shortest translation is about 17.36 A and the exact unique-image radius is about 8.68 A. The historical face-height bound was only about 7.09 A and therefore rejected a valid $r_{\max} = 8$ A RDF.

3.4 Cutoff inequality

A pair is a neighbor only when

$$r_{ij} < r_{\mathrm{cut}}.$$

The strict inequality is part of the internal contract. An atom exactly on the cutoff is excluded because such equality is numerically unstable and has zero measure in a continuous distribution.

3.5 Self pairs

Whenever center and neighbor selections overlap, pairs with

$$i = j$$

are excluded. The first implementation accepts selections that are either:

- identical; or
- disjoint.

Partially overlapping explicit selections are rejected because their pair normalization and duplicate-counting semantics are ambiguous.

4. Public cutoff objects

4.1 PairCutoff

```
@dataclass(frozen=True, slots=True)
class PairCutoff:
    atomic_numbers: tuple[int, int]
    radius: float
    source: Literal["manual", "rdf_first_minimum"] = "manual"
    source_metadata: Mapping[str, Any] = field(default_factory=dict)
```

The atomic-number pair is canonicalized:

$$(Z_a, Z_b) \mapsto (\min(Z_a, Z_b), \max(Z_a, Z_b)).$$

Therefore, `("Si", "O")` and `("O", "Si")` identify the same physical cutoff.

Required constraints:

- `radius` is finite and positive;
- both atomic numbers are valid elements;
- `source` is one of the documented values;
- RDF-derived cutoffs retain the minimum radius, feature confidence, smoothing settings, and source-frame provenance in `source_metadata`.

Recommended constructors:

```
PairCutoff.manual("Si", "O", radius=2.10)

PairCutoff.from_rdf_minimum(
    rdf_result,
    minimum_options={"smoothing_sigma": 0.05},
)
```

`PairCutoff.from_rdf_minimum()` calls `RDFResult.first_minimum()` and records an auditable cutoff provenance. The angle and coordination functions never run RDF minimum finding implicitly when a `PairCutoff` is already supplied. Each RDF selection must resolve to one unique chemical species. A mixed-species RDF may be analyzed normally, but it cannot be converted into one species-pair `PairCutoff`.

Implemented convenience interface:

```
cutoff.symbols
cutoff.matches("Si", "O")
cutoff.require_match("Si", "O")
payload = cutoff.to_dict()
cutoff = PairCutoff.from_dict(payload)
```

4.2 PairCutoffRegistry

```
@dataclass(frozen=True, slots=True)
class PairCutoffRegistry:
    cutoffs: Mapping[tuple[int, int], PairCutoff]
```

The registry is the common neighborhood definition for a structural analysis. It supports:

```
registry.require("Si", "O")
registry.get("Na", "O")
registry.contains("Al", "O")
registry.validate_for_collection(collection, frame_indices=...)
```

A convenience constructor may accept raw mappings:

```
registry = PairCutoffRegistry.from_mapping(
    {
        ("Si", "O"): 2.10,
        ("Al", "O"): 2.30,
        ("Na", "O"): na_o_cutoff,
    }
)
```

Raw floats are converted to manual `PairCutoff` values. Duplicate canonical pairs with inconsistent radii are rejected. Both cutoff provenance and the registry mapping are deeply immutable after construction, so connectivity and structural-analysis definitions cannot change silently after validation.

Implemented constructors and helpers:

```
PairCutoffRegistry.from_cutoffs([...])
PairCutoffRegistry.from_mapping({...})
registry.contains("Si", "O")
registry.get("Si", "O")
registry.require("Si", "O")
registry.validate_for_collection(collection, frame_indices=frames)
registry.to_dict()
```

`validate_for_collection()` checks every registered cutoff. Observable modules may instead validate only the pair cutoffs required by the requested analysis; unused registry entries do not affect such a calculation.

5. Internal result representation

5.1 `NeighborListResult`

One neighbor list represents one frame, one ordered center selection, one ordered candidate-neighbor selection, and one cutoff.

```
@dataclass(frozen=True, slots=True)
class NeighborListResult:
    frame_index: int
    center_indices: NDArray[np.int64]       # (N_c,)
    neighbor_indices: NDArray[np.int64]     # (N_p,)
    offsets: NDArray[np.int64]              # (N_c + 1,)
    vectors: NDArray[np.float64]            # (N_p, 3), A
    distances: NDArray[np.float64]          # (N_p,), A
    image_shifts: NDArray[np.int64]          # (N_p, 3)
    cutoff: float
    pair_counting: PairCounting
    backend: NeighborSearchBackend = NeighborSearchBackend.DENSE
```

The flattened arrays use compressed sparse row (CSR) grouping. Neighbors for local center slot `q` are

```
start = result.offsets[q]
stop = result.offsets[q + 1]
indices = result.neighbor_indices[start:stop]
vectors = result.vectors[start:stop]
distances = result.distances[start:stop]
image_shifts = result.image_shifts[start:stop]
```

Invariants:

- `offsets[0] == 0`;

- `offsets[-1] == n_pairs`;
- `offsets` is nondecreasing;
- all returned distances satisfy `distance < cutoff`;
- `distances == norm(vectors, axis=1)` within tolerance;
- each image shift reconstructs the same minimum-image vector through `vector = raw_displacement + image_shift @ cell`;
- image shifts are zero along nonperiodic axes;
- each group preserves deterministic candidate-selection order. Public RDF, coordination, and bond-angle callers canonicalize their atom selections, so their rows use increasing canonical atom-index order;
- all stored arrays are defensive copies and are read-only after construction;
- `backend` records which implementation produced the result without changing scientific equality.

Implemented row helpers:

```
result.n_centers
result.n_pairs
result.coordination_counts    # np.diff(result.offsets)
result.row_slice(local_center)
```

CSR storage is chosen because coordination and angle analysis group neighbors by center, while RDF can flatten the same result without rebuilding geometry.

5.2 Search backend

```python
class NeighborSearchBackend(str, Enum):
    DENSE = "dense"
    CELL_LIST = "cell_list"
    VERLET_CACHE = "verlet_cache"
```

`DENSE` is the blocked all-pairs oracle. `CELL_LIST` is the exact stage-S1 single-frame candidate generator. `VERLET_CACHE` is result provenance for `NeighborSearchSession`; it cannot be selected through the stateless facade. All three result paths return the same scientific arrays. The high-level S4 policy selects only dense or cell-list execution and activates a session only when the selected backend is cell list and the frame selection can reuse candidates.

The complete production contract is specified in `docs/specs/analysis/neighbor_search_spec.md`. The S1-S3 implementation details remain in the cell-list, fixed-cell cache, and deformation-aware cache specifications.

5.2.1 Cell-list options

```python
@dataclass(frozen=True, slots=True)
class CellListOptions:
    use_lattice_reduction: bool = True
    max_stencil_candidates: int = 1_000_000
    max_stencil_offsets: int = 250_000
    metric_tolerance: float = 1.0e-12
    coordinate_tolerance: float = 1.0e-12
    reduction_rtol: float = 1.0e-12
    reduction_atol: float = 1.0e-12
```

The options are immutable. The two stencil limits are hard safety limits; the implementation raises `CellListComplexityError` rather than truncating an exact stencil. Lattice reduction affects only internal candidate generation.

5.3 Pair-counting mode

```
class PairCounting(str, Enum):
    DIRECTED = "directed"
    UNORDERED_IDENTICAL = "unordered_identical"
```

`DIRECTED` means every center-to-neighbor relation is retained. It is required for coordination and angle analysis.

`UNORDERED_IDENTICAL` is valid only when the center and neighbor selections are identical. It retains one representative of each physical pair, using

$$i < j.$$

This mode is used by identical-species RDF calculations to avoid double counting.

6. Internal API

6.1 `build_neighbor_list`

```
def build_neighbor_list(
    collection: AtomisticFrameCollection,
    *,
    frame_index: int,
    center_indices: ArrayLike,
    candidate_neighbor_indices: ArrayLike,
    cutoff: float | PairCutoff,
    pair_counting: PairCounting = PairCounting.DIRECTED,
    backend: NeighborSearchBackend = NeighborSearchBackend.DENSE,
    block_size: int = 256,
    cell_list_options: CellListOptions | None = None,
) -> NeighborListResult:
    ...
```

The facade validates the requested backend and dispatches to either the dense oracle or the exact stage-S1 cell-list implementation. `block_size` is used by the dense backend. `cell_list_options` is used by the cell-list backend.

The dense routine:

1. validates the frame, selections, cell, PBC, cutoff, and block size;
2. obtains wrapped positions for the frame;
3. processes center atoms in blocks;
4. computes minimum-image vectors, distances, and integer image shifts to every candidate neighbor;
5. applies self-pair and pair-counting rules;
6. applies the strict cutoff;
7. writes a deterministic, immutable CSR result.

The cell-list routine:

1. validates the same scientific inputs as the dense routine;
2. optionally constructs a Minkowski-reduced internal search basis;
3. partitions wrapped search-basis fractional coordinates into linked cells;
4. constructs an exact metric-aware bin-offset stencil;
5. gathers and deduplicates candidate atoms from reachable bins;
6. restores caller candidate-selection order;
7. recomputes exact minimum-image geometry in the original cell basis;

8. applies the same self, pair-counting, coincidence, and strict-cutoff rules;
9. writes the same deterministic, immutable CSR result.

6.2 `iter_neighbor_lists`

```
def iter_neighbor_lists(
    collection: AtomisticFrameCollection,
    *,
    frame_indices: ArrayLike,
    center_indices: ArrayLike,
    candidate_neighbor_indices: ArrayLike,
    cutoff: float | PairCutoff,
    pair_counting: PairCounting = PairCounting.DIRECTED,
    backend: NeighborSearchBackend = NeighborSearchBackend.DENSE,
    block_size: int = 256,
    cell_list_options: CellListOptions | None = None,
) -> Iterator[NeighborListResult]:
    ...
```

The iterator avoids storing neighbor geometry for all frames simultaneously. Observable modules reduce each frame result immediately.

6.3 Geometry helpers

```
def minimum_image_geometry(
    displacements: ArrayLike,
    *,
    cell: ArrayLike,
    pbc: ArrayLike,
) -> tuple[FloatArray, FloatArray, IntArray]:
    ...
```

Returns minimum-image vectors, distances, and integer lattice-image shifts for arbitrary batched Cartesian displacements. The image shift `m` satisfies

$$\mathbf{d}_{\mathrm{MIC}} = \mathbf{d}_{\mathrm{raw}} + \mathbf{m}H.$$

The existing compatibility helper remains:

```
def minimum_image_vectors(
    displacements: ArrayLike,
    *,
    cell: ArrayLike,
    pbc: ArrayLike,
) -> tuple[FloatArray, FloatArray]:
    ...
```

It returns only vectors and distances for observable modules that do not need periodic graph information.

```
def compute_safe_cutoff(
    collection: AtomisticFrameCollection,
    *,
    frame_indices: ArrayLike,
) -> float:
    ...
```

Returns the exact global unique minimum-image radius over the selected frames. For each frame this is one half of the shortest nonzero periodic lattice translation, obtained from a validated Minkowski-reduced basis.

```
def validate_cutoff(
    cutoff: float | PairCutoff,
    *,
    collection: AtomisticFrameCollection,
    frame_indices: ArrayLike,
) -> float:
    ...
```

Returns the numeric radius after validating positivity and the global minimum-image-safe limit.

7. Dense-oracle comparison infrastructure

Stage S0 adds private backend-verification helpers in

`mdstats/analysis/_neighbor_compare.py`

The helpers are not scientific APIs. They exist so every optimized backend can be tested against the dense reference with one exact comparison contract.

```
canonicalize_neighbor_result(result)
compare_neighbor_results(actual, expected, options=...)
assert_neighbor_results_equal(actual, expected, options=...)
```

Canonicalization sorts center rows by canonical atom index and sorts each row by

`(neighbor atom index, image_shift_x, image_shift_y, image_shift_z)`

while moving vectors and distances with the same pair record. Comparison checks:

- frame and pair-counting metadata;
- cutoff agreement;
- canonical center indices;
- CSR offsets;
- canonical neighbor indices;
- exact integer image shifts;
- tolerance-bounded Cartesian vectors and distances.

Backend metadata is ignored by default because a future optimized result should be scientifically equal to a dense result even though its implementation label differs. Tests may request strict backend comparison explicitly.

The randomized test harness uses stored seeds and an independent scalar pair loop. It covers orthogonal, triclinic, mixed-PBC, and boundary-crossing cases, with both directed-disjoint and unordered-identical selections. Dense results must agree across block sizes and repeated runs. Stage S1 additionally requires cell-list results to agree with the dense oracle in pair identity, CSR grouping, and original-basis image shift, and within numerical tolerance in vectors and distances.

8. Selection and combined-species semantics

The shared kernel receives explicit canonical atom-index arrays. Public modules resolve species selections before calling it.

For a combined species set S, candidate atom indices are the set union

$$I_S = \bigcup_{X \in S} I_X.$$

The union is deduplicated and sorted. Combined coordination is therefore computed from a true atom set, not by adding separately calculated counts. This definition remains correct even when future explicit selections overlap.

Each species pair in a combined condition may require a different cutoff. The caller builds one neighbor list per center-neighbor pair and forms the union of accepted canonical neighbor indices before counting.

9. RDF integration

`compute_pair_rdf()` uses `UNORDERED_IDENTICAL` when the two atom selections are identical and `DIRECTED` when they are disjoint.

The neighbor cutoff is `r_max`, not a chemical-bond cutoff. RDF consumes only `distances`; it applies shell binning and framewise density normalization itself.

The shared layer does not calculate:

- shell volumes;
- ideal-gas normalization;
- cumulative coordination;
- smoothing;
- peak or minimum detection.

10. Coordination integration

`compute_coordination_distribution()` uses `DIRECTED` lists. For every center, coordination is

$$n_i(t) = \texttt{offsets}[i+1] - \texttt{offsets}[i].$$

The authoritative integer matrix is assembled from these row lengths. The coordination module remains responsible for distributions, means, variances, and RDF-cutoff provenance.

11. Bond-angle integration

For triplet $A - B - C$, the angle module builds directed neighbor lists centered on B for pairs $B - A$ and $B - C$.

For each accepted center:

- if $A = C$, use unordered pairs from one neighbor set;
- if $A \neq C$, use the Cartesian product of the two neighbor sets.

The stored displacement vectors are reused directly in

$$\theta = \cos^{-1}\left(\frac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|}\right).$$

No second minimum-image calculation is performed.

12. Complexity and memory

For N_c centers and N_n candidate neighbors in one frame, the blocked dense oracle has worst-case work

$$O(N_c N_n).$$

Its temporary memory is approximately

$$O(BN_n),$$

where B is the center block size.

For the cell-list backend, let S be the retained bin-stencil size and let z_{bin} be the average number of unique atom candidates gathered from visited bins. The approximate work is

$$O(N_c + N_n) + O(N_c S) + O(N_c z_{\mathrm{bin}}).$$

At fixed density and finite cutoff, S and z_{bin} are expected to remain bounded, giving approximately linear scaling. The worst case remains quadratic when bins or the stencil contain most atoms.

Both backends store final CSR output in

$$O(N_p),$$

where N_p is the accepted-pair count.

The dense baseline is `benchmarks/dense_neighbors_benchmark.md`. The S1 equivalence and machine-specific benchmark is `benchmarks/cell_list_benchmark.md`.

13. Verlet candidate caching

Stages S2-S3 add `NeighborSearchSession`, `VerletCacheOptions`, `VerletPairCache`, and `NeighborCacheStatistics`.

For one request with physical cutoff r_c and skin r_s, a rebuild uses the exact S1 cell list at

$$R = r_c + r_s.$$

The default stage-S2 policy requires an unchanged cell and

$$2d_{\max} < r_s - \varepsilon.$$

Stage S3 is explicit through:

```
VerletCacheOptions(deformation_aware=True)
```

Let the rebuild and current cells be H_0 and H_t. Define

$$F_t = H_0^{-1} H_t.$$

For active species pair (A, B), the cache remains complete only while

$$M_{AB}(t) = \sigma_{\min}(F_t)(r_{AB} + r_s) - r_{AB} - u_A^{\max}(t) - u_B^{\max}(t) > \varepsilon.$$

The nonaffine displacement is derived from continuous trajectory fractional coordinates:

$$\mathbf{u}_i(t) = [\mathbf{s}_i(t) - \mathbf{s}_i(t_0)]H_t.$$

Active pair types are derived from the exact center/candidate selections. Same-species pairs are included only when two distinct selected atoms can form that pair. This avoids doubling the displacement of a singleton mobile species.

Rigid cell rotations have $\sigma_{\min} = 1$ and do not rebuild when fractional coordinates are unchanged. Singular or over-conditioned cells raise `InvalidCellGeometryError`. Independent ensembles use the S2 MIC bound when the cell is unchanged and rebuild when the cell changes because a continuous fractional unwrap is unavailable.

Current MIC vectors, distances, and image shifts are recomputed for cached pairs on every frame. The request digest includes atom selections, pair-counting mode, cutoff, skin, PBC and atomic-number schema, cell-list options, deformation policy, and condition-number limit.

The session is request-keyed and not thread-safe. It stores one cache per exact request and exposes rebuild, reuse, candidate, accepted-pair, and reason statistics. The stateless `build_neighbor_list()` and `iter_neighbor_lists()` functions remain uncached.

RDF, coordination, bond-angle, and every distance-based atomic-connectivity mode construct one analysis-local S4 executor. Atomic connectivity retains `verlet_cache_options` only as a compatibility alias; new code uses `neighbor_search_options`. Hysteretic and reference bond state remains owned by atomic connectivity, not by the cache.

The complete S2 and S3 contracts are specified in:

`docs/specs/analysis/_verlet_cache_spec.md`
`docs/specs/analysis/_verlet_cache_deformation_spec.md`

14. Validation and exceptions

Recommended internal exceptions:

```
class NeighborError(RuntimeError): ...
class InvalidNeighborSelectionError(NeighborError): ...
class InvalidNeighborCutoffError(NeighborError): ...
class UnsafeNeighborCutoffError(NeighborError): ...
class InvalidCellGeometryError(NeighborError): ...
class CoincidentAtomsError(NeighborError): ...
class CellListComplexityError(NeighborError): ...
```

The layer rejects:

- empty selections;
- out-of-range or duplicate explicit indices;
- partially overlapping pair selections;
- nonfinite or singular cells;
- nonfinite coordinates;
- nonpositive cutoffs;
- cutoffs beyond the safe minimum-image radius;
- invalid pair-counting modes;
- zero or negative block sizes.

Coincident distinct atoms produce a zero-length vector. Coordination and RDF may still count such a pair mathematically, but angle calculation cannot use it. The shared layer records or raises a dedicated diagnostic so the caller cannot silently create `NaN` angles.

15. Testing requirements

Focused tests must cover:

1. Orthogonal minimum-image vectors.
2. Restricted and general triclinic cells.
3. Mixed periodic and nonperiodic axes.
4. Strict exclusion at `distance == cutoff`.
5. Self-pair exclusion.
6. Directed counting for identical selections.
7. Unordered identical-pair counting.
8. CSR offsets and deterministic ordering.
9. Immutable result arrays.
10. Explicit dense- and cell-list-backend selection.
11. Dense agreement across center block sizes.
12. Repeated-run determinism for both backends.
13. Canonical comparison under permuted selection order.
14. Exact image-shift mismatch diagnostics.
15. Stored-seed randomized agreement with a scalar pair loop.
16. Cell-list equivalence for highly skewed cells with reduction on and off.
17. Fully periodic, mixed-PBC, one-dimensional periodic, and nonperiodic cases.
18. Zero/one-pair, dense-cluster, and near-safe-cutoff cases.
19. Multiple species-pair selections and cutoffs.
20. Original-basis image-shift preservation after search-basis reduction.
21. Cell-list option validation and stencil hard limits.
22. Fixed-cell cache reuse against fresh dense and cell-list results.
23. Exact displacement-threshold rebuilds and periodic boundary crossings.
24. Request-keyed invalidation and conservative default cell-change rebuilds.
25. Deformation-aware isotropic, orthorhombic, shear, and rigid-rotation reuse.
26. Affine and nonaffine margin-threshold rebuilds.
27. Species-aware displacement maxima and singleton mobile-species handling.
28. Periodic boundary crossings during variable-cell motion.
29. Adversarial omitted-pair completeness near the theoretical margin.
30. Explicit ill-conditioned-cell rejection.
31. Cached hysteretic and reference connectivity equivalence.
32. Combined-species unions without duplicate atoms.
33. Exact shortest-translation safe-cutoff validation over variable cells.
34. Skewed LTA primitive cells for which 8 A is valid but the historical face-height bound is smaller.
35. Partial-periodic shortest translations and periodic-sublattice preservation.
36. PairCutoff canonicalization and registry conflicts.
37. Agreement of RDF and coordination counts from the same neighbor lists.

16. Deliberate non-goals

The production neighbor subsystem does not implement:

- dynamic bond lifetimes;
- hysteretic bond definitions inside the neighbor kernel;
- Voronoi or Delaunay neighbors;
- fixed-nearest-neighbor lists;
- bond-order criteria;
- automatic RDF minimum selection;
- GPU acceleration.

The exact cell-list backend, fixed- and variable-cell cache, deterministic automatic policy, semantics-aware cache activation, runtime zero-reuse shutoff, consumer integration, and diagnostic provenance are implemented. Any later backend must remain behind the same result contract.

17. Reproduction checklist

An independent implementation reproduces this design when it:

1. uses the ASE row-vector cell convention;
2. computes general-cell minimum-image displacement vectors;
3. applies `distance < cutoff`;
4. excludes self pairs;
5. distinguishes directed and unordered-identical pair counting;
6. returns deterministic CSR grouping by center;
7. validates all cutoffs against half the exact shortest nonzero periodic lattice translation over the selected frames;
8. keeps RDF, coordination, and angle normalization outside the shared layer;
9. preserves cutoff provenance through `PairCutoff` and `PairCutoffRegistry`;
10. exposes the blocked dense implementation explicitly as the reference backend;
11. exposes the exact single-frame cell-list backend only by explicit request;
12. uses perpendicular-height fractional bins and a metric-aware stencil;
13. computes final image shifts in the original cell basis;
14. can canonicalize and compare results by CSR grouping, pair identity, image shift, vector, and distance;
15. can store continuous fractional rebuild references;
16. evaluates the smallest singular value of $H_0^{-1}H_t$;
17. computes species-resolved nonaffine maxima and all active pair margins;
18. rejects singular or over-conditioned deformation cells;
19. preserves exact dense and fresh-cell-list equality on every cached frame.

18. Stage S4 policy delegation

Scientific neighbor semantics remain owned by this document and `_neighbors.py`. Execution policy, cache activation, fallback, and unified diagnostics are owned by:

```
docs/specs/analysis/neighbor_search_spec.md
mdstats/analysis/neighbor_search.py
```

The default automatic policy estimates dense pair work, uses the conservative threshold `32768`, and otherwise selects the exact cell list. It never selects `VERLET_CACHE` as a stateless backend; cache reuse is an execution mode inside the cell-list path.

Cache activation is resolved once per normalized request from the selected-frame semantics:

```
single selected frame -> stateless
multi-frame trajectory -> eligible for automatic Verlet reuse
independent ensemble -> stateless by default
```

Explicit `cache_mode="verlet"` remains an expert override for geometrically related ensembles. Any active cache is disabled for the rest of the request after three consecutive completed rebuild intervals with no successful reuse; one reuse resets the counter. This policy changes performance only. Exact neighbor results remain governed by the same dense/cell-list/cache contract.

19. References

1. Larsen, A. H., et al. (2017). *The Atomic Simulation Environment - A Python Library for Working with Atoms*. Journal of Physics: Condensed Matter, 29(27), 273002. DOI: 10.1088/1361-648X/aa680e.
2. Nguyen, P. Q., and Stehle, D. (2009). *Low-Dimensional Lattice Basis Reduction Revisited*. ACM Transactions on Algorithms, 5(4), Article 46. DOI: 10.1145/1597036.1597050.